

CS-112

Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

COURSE INSTRUCTOR : MS. HABIBA ARSHAD

Topics To be Covered Today

- ☐ Variables
- ☐ Scope and Lifetime of variable
- ☐ Constants
- ☐ Arithmetic Expression
- ☐ Java Classes
- ☐ Control Structure



Variables

Variable is a named location where data can be stored. It is a location in computer's memory with a specific address, where a value can be stored and retrieved when required.

A variable is assigned with a data type.

```
int data=50; //Here data is variable
```

Types:

There are three types of variables in java:

- local variable
- instance variable
- static variable

Variables Declaration and initialization in Java

Format:

<data type> <name of variable>;

Example:

char var1;

Variables can be initialized (set to a starting value) as they're declared:

char var1 = 'j';

int age = 30;

float x=3.1f;

double y=3.145d;

Local Variables

A variable declared inside the body of the method is called local variable.

You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

Create multiple copies each time when the class object is created. Means it does NOT retain its value across method calls (it gets reinitialized each time).

Static Variable

A variable that is declared using static keyword at the class level.

It cannot be local.

Only single copy of the static or class variable is created and shared among all the instances of the class.

Memory allocation for static variables happens only once when the class is loaded in the memory.

They are created when the class is loaded and destroyed when it is unloaded.

Static vs Local Variables

```
Class A{  
    static int a = 10; // static variable  
    void method(){  
        int b = 10; // local variable  
        system.out.println(a+ " " +b);  
        ++a;  
        ++b;  
    }  
}
```

```
public static void main (String [] args)  
{  
    A ref = new A();  
    ref.method();  
    ref.method();  
}  
}
```

Instance Variable

A variable declared inside the class but outside the body of the method, is called an instance variable.

It is not declared as static.

It is called an instance variable because its value is instance-specific and is not shared among instances.

Create multiple copies each time when the class object is created.

Example to understand the types of variables in java

```
public class A {  
    static int m=100;//static variable  
    int n=90;//instance variable  
    public static void main(String[] args)  
    {  
        int data=50;//local variable  
        System.out.println(data);  
        System.out.println(A.m); // Static variables are class members directly  
                                   call through class name no need for object  
        A ref = new A();  
        System.out.println(ref.n); //To access the instance variable object is required.  
    } }//end of class
```

//Write a program to calculate the average of three numbers

```
1 class average
```

```
2 {
```

```
3     public static void main(String arg[])
```

```
4     {
```

```
5         int a=10, b=14, c=33;
```

```
6         float avg;
```

```
7         avg= (a+b+c)/3; //Dynamic initialization of variable avg
```

```
8         System.out.println("The average of three numbers is: " +avg);
```

```
9     }
```

```
10 }
```

Scope and Lifetime of Variable

Scope of a variable determines the area or a region of code where a variable is available to use.

Lifetime of a variable is defined by the time for which a variable occupies some valid space in the system's memory.

Summary of scope and lifetime of variables

Variable Type	Scope	Lifetime
Instance variable	Throughout the class except in static methods	Until the object is available in the memory
Class variable	Throughout the class	Until the end of the program
Local variable	Within the block in which it is declared	Until the control leaves the block in which it is declared

Example

```
public class EggBasket
{
    public static void main(String[] args)
    {
        int numberOfBaskets, eggsPerBasket, totalEggs;
        numberOfBaskets = 10;
        eggsPerBasket = 6;

        totalEggs = numberOfBaskets * eggsPerBasket;

        System.out.println("If you have");
        System.out.println(eggsPerBasket + " eggs per basket and");
        System.out.println(numberOfBaskets + " baskets, then");
        System.out.println("the total number of eggs is " + totalEggs);
    }
}
```

Variable
declarations

Assignment statement

Sample Screen Output

If you have
6 eggs per basket and
10 baskets, then
the total number of eggs is 60

Java Class Libraries

The Java environment relies on several built-in class libraries that contain many built-in methods that provide support for such things as I/O, string handling, networking, and graphics.

The standard classes also provide support for windowed Input/output.

Thus, Java as a totality is a combination of the Java language itself, plus its standard classes.

Java (class Scanner)

- We use the class Scanner, which Java supplies, to accept keyboard input.
- Our program must import the definition of the Scanner class from the package `java.util`. Thus, we begin the program with the following statement:

```
import java.util.Scanner;
```

The following line sets things up so that data can be entered from the keyboard:

```
Scanner keyboard = new Scanner(System.in);
```

This line must appear before the first statement that takes input from the keyboard. That statement in our example is

```
eggsPerBasket = keyboard.nextInt();
```

This assignment statement gives a value to the variable `eggsPerBasket`. The expression on the right side of the equal sign, namely

```
keyboard.nextInt()
```

```
import java.util.Scanner;
```

Gets the Scanner class from the package (library) java.util

```
public class EggBasket2
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int numberOfBaskets, eggsPerBasket, totalEggs;
```

Sets up things so the program can accept keyboard input

```
        Scanner keyboard = new Scanner(System.in);
```

```
        System.out.println("Enter the number of eggs in each basket:");
```

```
        eggsPerBasket = keyboard.nextInt();
```

Reads one whole number from the keyboard

```
        System.out.println("Enter the number of baskets:");
```

```
        numberOfBaskets = keyboard.nextInt();
```

```
        totalEggs = numberOfBaskets * eggsPerBasket;
```

```
        System.out.println("If you have");
```

```
        System.out.println(eggsPerBasket + " eggs per basket and");
```

```
        System.out.println(numberOfBaskets + " baskets, then");
```

```
        System.out.println("the total number of eggs is " + totalEggs);
```

```
        System.out.println("Now we take two eggs out of each basket.");
```

```
        eggsPerBasket = eggsPerBasket - 2;
```

```
        totalEggs = numberOfBaskets * eggsPerBasket;
```

```
        System.out.println("You now have");
```

```
        System.out.println(eggsPerBasket + " eggs per basket and");
```

```
        System.out.println(numberOfBaskets + " baskets.");
```

```
        System.out.println("The new total number of eggs is " + totalEggs);
```

```
    }
```

```
}
```

```
import java.util.Scanner;

public class UserInputExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Taking different types of input from user
        System.out.print("Enter a byte value: ");
        byte byteValue = scanner.nextByte();

        System.out.print("Enter a character: ");
        char charValue = scanner.next().charAt(0);

        System.out.print("Enter an integer: ");
        int intValue = scanner.nextInt();

        System.out.print("Enter a long value: ");
        long longValue = scanner.nextLong();

        System.out.print("Enter a float value: ");
        float floatValue = scanner.nextFloat();
    }
}
```

```
System.out.print("Enter a double value: ");
double doubleValue = scanner.nextDouble();

scanner.nextLine(); // Consume newline left-over

System.out.print("Enter a string: ");
String stringValue = scanner.nextLine();

// Displaying user inputs
System.out.println("\nYou entered the following
values:");
System.out.println("Byte: " + byteValue);
System.out.println("Character: " + charValue);
System.out.println("Integer: " + intValue);
System.out.println("Long: " + longValue);
System.out.println("Float: " + floatValue);
System.out.println("Double: " + doubleValue);
System.out.println("String: " + stringValue);

scanner.close(); // Closing scanner
```

Method	Purpose	Input Example	Why Use It?
<code>nextByte()</code>	Reads a <code>byte</code>	<code>10</code>	For small integer values (-128 to 127)
<code>charAt(0)</code>	Extracts a character	<code>"A"</code>	Gets the first character of a string
<code>nextInt()</code>	Reads an <code>int</code>	<code>1000</code>	For whole numbers
<code>nextLong()</code>	Reads a <code>long</code>	<code>9876543210</code>	For large whole numbers
<code>nextFloat()</code>	Reads a <code>float</code>	<code>3.14</code>	For decimal numbers with less precision
<code>nextDouble()</code>	Reads a <code>double</code>	<code>9.87654321</code>	For decimal numbers with high precision
<code>nextLine()</code>	Reads a full line	<code>"Hello, World!"</code>	For complete sentences

Constants

Constants are like variables their value CANNOT change.

If the constant is referred to several times throughout the program, changing the value of the constant once will change it throughout the program.

Format:

```
final <type ><CONSTANT NAME> = <value>;
```

Example:

```
final int var = 100;
```

```
final double pi= 3.14159;
```

```
private static final double pi= 3.14159;
```

Arithmetic Expression Evaluation

To evaluate an arithmetic expression two concepts need to be understood

- **Operator Precedence**
 - Operator precedence controls the order in which operations are performed
- **Operator Associativity**
 - The associativity of an operator specifies the order in which operations of the same precedence are performed

Operator Precedence and Associativity

Operators Precedence and Associativity for Java is following

1. $*, /, \%$ Do all multiplications, divisions and remainders from left to right.
2. $+, -$ Do additions and subtractions from left to right.

Operators	Symbols	Associativity
Parenthesis	(), { }, []	Left to Right
Exponents	$\wedge$	Right to Left
Multiplication and Division	$*, /$	Left to Right
Addition and Subtraction	$+, -$	Left to Right

Evaluating an Expression

$$6 + 2 * 3 / 6$$

Three operators are in this expression.

However, $*$ and $/$ both have the same precedence and $+$ has lower precedence than these two.

$*$ and $/$ will be evaluated first but both have the same precedence level.

Therefore, operator associativity will be used here to determine the first to get evaluated i.e. left to right.

The right most sub expression will be evaluated followed by the next right one and so on.

Class Participation

What will happen?

```
float f=65/10+38/10;  
System.out.println(f);
```



Task

List down Java Compile –time and Run-time errors.